

The Spence Loss: Well-Conditioned Value-Function Learning for Reward-Guided Diffusion

David Li-Bland ...

Abstract

Reward-guided diffusion and diffusion fine-tuning are, through entropy-regularized stochastic control, the problem of learning a value function $V(x, t) = \log \mathbb{E}_{\text{base}}[\exp r(X_1) \mid X_t = x]$ —a conditional log-partition function whose gradient is the optimal control. The unbiased Monte-Carlo target for V lives in exponential space, so squared-error regression has gradients that explode for over-predictions and, worse, vanish for under-predictions, where the model is most wrong and gradient clipping cannot help. We observe that every Bregman divergence has the value function as its population minimizer, and that such a loss reaches the optimizer only through a scalar *gradient weight* $w(u) = u\varphi''(u)$ on the exponential-space residual—so the design space is exactly the choice of w , with squared error ($w = 2u$) and the Itakura–Saito divergence ($w = 1/u$) as its two extremes. We then solve for the weight $w(u) = \ln u/(u-1)$ whose gradient is linear in the log-space prediction in *both* tails, yielding the *Spence loss*¹: a closed-form, numerically stable gradient under which neither over- nor under-predictions blow up or die. A secondary contribution analyzes why high-dimensional value learning is signal-sparse and studies on-policy importance-sampling schemes. We validate on 2-moons, a dimensional-scaling benchmark designed for clean statistical reads, and GMM-40, where our log-partition estimate is competitive with published diffusion samplers.

1 Introduction

2 Setting: value functions for reward-guided diffusion

2.1 The base process and the tilted target

Fix a time horizon $t \in [0, 1]$ and a base diffusion on $\mathbb{R}^d$,

$$dX_t = f(X_t, t) dt + \sqrt{2a} dW_t, \quad X_0 = 0, \quad (1)$$

with $a > 0$ a scalar diffusion coefficient and W a standard d -dimensional Brownian motion. Write $\mathbb{P}$ for the induced path measure, $\mathbb{E}_{\text{base}}$ and p_{base} for expectations and marginals under it, and $\mathcal{L} = f \cdot \nabla + a\Delta$ for its generator. Given a terminal reward $r : \mathbb{R}^d \rightarrow \mathbb{R}$, the object we wish to sample from is the *reward-tilted* path measure

$$\frac{d\mathbb{P}^*}{d\mathbb{P}} = \frac{e^{r(X_1)}}{Z}, \quad Z = \mathbb{E}_{\text{base}}[e^{r(X_1)}], \quad (2)$$

whose terminal marginal is $p^*(x) \propto p_{\text{base}}(x) e^{r(x)}$. This is the common form of reward-guided generation and of diffusion fine-tuning [3], and it also covers Boltzmann sampling: to target $\pi \propto e^{-E}$ one sets $r = -E - \log p_{\text{base}}$, which is how we instantiate the GMM-40 benchmark in §7.

Nothing below depends on f having a particular form: if $\mathbb{P}$ is a pretrained diffusion or flow model, f is its learned drift and the whole development applies unchanged.²

¹Named for the Spence (dilogarithm) function Li_2 appearing in its value; in our code and experiments it is the **quad** loss.

²Note the time convention: t runs from a deterministic start $X_0 = 0$ to the model’s samples at $t = 1$, as in the diffusion-sampler literature [5, 4], rather than the denoising direction of DDPM-style presentations.

The base family we use. Our base processes are *pinned interpolants*: $X_0 = 0$, a prescribed terminal marginal ν , and a Brownian bridge between the two given $X_1 = x_1$. The induced drift $f(x, t) = \mathbb{E}[(X_1 - X_t)/(1 - t) \mid X_t = x]$ vanishes exactly when $\nu = \mathcal{N}(0, 2aI)$, recovering driftless Brownian motion (the case used for the Boltzmann benchmarks); taking ν to be a Gaussian mixture gives a base model with nontrivial but analytically known f , V and $\log Z$, which is the controlled setting of our dimensional-scaling study (§7). The family is convenient because it makes exact ground truth and a closed-form bridge available at once (§2.4).

2.2 Control formulation and the value function

Realizing (2) means adding a drift. For a control u giving the controlled dynamics $dX_t = (f + u) dt + \sqrt{2a} dW_t$ with path measure $\mathbb{P}^u$, Girsanov’s theorem gives $\text{KL}(\mathbb{P}^u \parallel \mathbb{P}) = \mathbb{E}_{\mathbb{P}^u} \int_0^1 \frac{1}{4a} \|u_t\|^2 dt$, so (2) is the solution of the entropy-regularized control problem

$$\max_u \mathbb{E}_{\mathbb{P}^u} \left[r(X_1) - \int_0^1 \frac{\|u_t\|^2}{4a} dt \right], \quad (3)$$

whose optimum is $\log Z$. The central object is the *soft value function*

$$V(x, t) = \log \mathbb{E}_{\text{base}}[e^{r(X_1)} \mid X_t = x], \quad (4)$$

a conditional log-partition function. It solves the HJB equation $\partial_t V + f \cdot \nabla V + a\Delta V + a\|\nabla V\|^2 = 0$ with terminal condition $V(\cdot, 1) = r$, the optimal control is

$$u^*(x, t) = 2a \nabla_x V(x, t), \quad (5)$$

and, since $X_0 = 0$ is deterministic, $V(0, 0) = \log Z$ — the normalizing constant, which is the headline metric on external sampler benchmarks (§7). Learning V therefore solves the sampling problem, the fine-tuning problem, and the free-energy-estimation problem at once, and everything downstream reduces to *regressing V from Monte-Carlo targets*.

2.3 The H-martingale property

Exponentiating (4) linearizes the HJB equation. Writing $h = e^V$, Feynman–Kac gives $\partial_t h + \mathcal{L}h = 0$: h is space-time harmonic for the base process, so $h(X_t, t)$ is a $\mathbb{P}$ -martingale, and (5) is exactly the Doob h -transform drift $f + 2a\nabla \log h$. Concretely, for any $0 \leq t < s \leq 1$,

$$e^{V(x, t)} = \mathbb{E}_{\text{base}}[e^{V(X_s, s)} \mid X_t = x] \quad (6)$$

with the terminal case $V(\cdot, 1) = r$. We call (6) the H-martingale property; it is the source of every target in this paper, and two of its features drive the design.

(i) Targets are unbiased in exponential space — and only there. Equation (6) says that a *single* draw of X_s from the base kernel supplies a scalar $q = V(X_s, s)$ (or $q = r(X_1)$ when $s = 1$) satisfying

$$\mathbb{E}[e^q \mid X_t = x] = e^{V(x, t)}. \quad (7)$$

The estimator is unbiased for e^V , not for V : the natural log-space plug-ins are both biased low by a Jensen gap (§3). This is a property of the log-partition function rather than of a particular scheme, and it is what forces the regression into exponential space — and hence into the conditioning problem of §3–§4.

(ii) The identity is twist-independent. Suppose we do not want to draw X_s from the base kernel — for instance because base samples rarely reach high-reward regions (§6). Propose instead under any adapted twist u , giving the drift $f + u$ and path measure $\mathbb{P}^u$, and correct by the Radon–Nikodym derivative of the base law with respect to it. By Girsanov’s theorem, restricted to the interval $[t, s]$,

$$\rho_{t \rightarrow s} = \frac{d\mathbb{P}}{d\mathbb{P}^u} \Big|_{[t, s]} = \exp \left(-\frac{1}{2a} \int_t^s u_\tau \cdot dB_\tau - \frac{1}{4a} \int_t^s \|u_\tau\|^2 d\tau \right), \quad (8)$$

where $dB_\tau = \sqrt{2a} dW_\tau^u$ is the driving noise under $\mathbb{P}^u$ (so the stochastic integral is an Itô integral against the noise actually realized by the sampler). Under Novikov’s condition $\rho_{t \rightarrow s}$ is an exponential martingale with unit mean, and (6) becomes

$$e^{V(x,t)} = \mathbb{E}_{\mathbb{P}^u}[\rho_{t \rightarrow s} e^{V(X_s,s)} \mid X_t = x]. \quad (9)$$

In an Euler–Maruyama discretization (8) factorizes over steps, each contributing $-u \cdot \Delta B / (2a) - \|u\|^2 \Delta t / (4a)$ with $\Delta B \sim \mathcal{N}(0, 2a \Delta t I)$; this per-step ratio is the incremental weight used in twisted-SMC samplers [2]. The twist u is therefore *free*: it changes the variance of the target, never its mean. The proposal is a variance-reduction device, not a modeling choice — which is what licenses the importance-sampling and SMC machinery of §6 without changing the object being learned.

2.4 How training pairs are formed

A learner V_θ is fit to pairs (x_t, q) . It is worth separating two kinds. *Anchored* targets terminate at the true reward and satisfy (7) exactly, so e^q is unbiased for $e^{V(x_t,t)}$ however wrong the current V_θ is. *Bootstrapped* targets substitute the learner’s own value at a later time, and satisfy no such identity. Both appear below; the distinction matters, and we are explicit about which guarantee is in force.

Off-policy (bridge) anchors. Draw a terminal point $x_1 \sim \nu$ from the base terminal marginal, place x_t on the base bridge to it, and take $q = r(x_1)$. For the pinned interpolants of §2 that bridge is the Brownian bridge in closed form,

$$x_t = t x_1 + \sqrt{2at(1-t)} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (10)$$

so the sampled joint (x_t, x_1) is *exactly* the base joint and q is anchored in the sense above. What matters is that x_1 is drawn from the base marginal ν : anchoring on a reward-aware or held-out data distribution instead breaks the conditional and biases the targets.

These anchors are cheap *per sample* — no trajectory is integrated, and each pair costs a single evaluation of r — and they cover the whole marginal. Two caveats. First, x_1 is drawn without regard to r , which is what fails in high dimension (§6). Second, cheap per sample is not the same as cheap in total: because the anchors are uninformative about where r is large, reaching a given accuracy can take many more of them, and hence many more reward evaluations, than a targeted scheme. When r is itself expensive — a quantum-chemistry calculation, a docking score, a simulator rollout — total calls to r , not wall-clock per sample, is the budget that binds, and the ranking between the constructions can invert. Our benchmarks all have cheap analytic rewards, so we report sample counts rather than reward-call counts; we flag this as a limitation in §9.

On-policy (bootstrapped) targets. Roll a sequential-Monte-Carlo sweep forward from $X_0 = 0$ under the current twist $u = 2a \nabla V_\theta$. A particle at (x_t, t) spawns k children at $s = t + \Delta t$ under the guided kernel; writing ρ_i for the per-step ratio of (8) and $v_i = V_\theta(x_s^{(i)}, s)$, the target attached to the parent is the log-mean-exp over its siblings,

$$q = \log \left(\frac{1}{k} \sum_{i=1}^k \rho_i e^{v_i} \right). \quad (11)$$

It is worth being exact about what this estimates. Define the one-step backup operator $\mathcal{T}_s$ acting on a candidate value function W by

$$(\mathcal{T}_s W)(x, t) = \log \mathbb{E}_{\text{base}}[e^{W(X_s,s)} \mid X_t = x]. \quad (12)$$

The children are i.i.d. from the guided kernel given the parent, so by (9)

$$\mathbb{E}[e^q \mid x_t] = e^{(\mathcal{T}_s V_\theta)(x_t, t)}. \quad (13)$$

That is the whole guarantee: (11) is an exp-space unbiased estimate of the backup of *the current network*, and of nothing else. In particular it is not an estimate of $V(x_t, t)$. Because e^{V_θ} is not harmonic, the children’s values need not be good estimates of the true value at the parent, and whatever error V_θ carries at time s is inherited rather than corrected. What keeps the recursion honest is the terminal step, where the children carry $q = r(X_1)$ exactly; the scheme is useful to the extent that this anchoring propagates backwards. By (6) the true value is a fixed point, $\mathcal{T}_s V = V$, but we prove nothing about convergence to it and treat the bootstrap as an empirical device throughout.

Backward-noising expansion. Because e^V is harmonic, a target attached at (x_s, s) remains valid at any point noised *backwards* from it under the base backward kernel. For the pinned interpolants this kernel is available in closed form and, notably, does not depend on the terminal law ν at all:

$$X_t | X_s = x_s \sim \mathcal{N}\left(\frac{t}{s} x_s, 2a \frac{t(s-t)}{s} I\right), \quad t < s, \quad (14)$$

because conditioning a Brownian bridge on an interior point leaves a Brownian bridge to that point, whatever the endpoint distribution. The expansion is therefore available for base models with nontrivial drift, not only for Brownian motion. Since x_s came from the base marginal and x_t from the base backward kernel, the pair (x_t, q) carries the base joint law and (6) applies unchanged. This turns one trajectory into many training pairs at earlier, noisier times, and is the mechanism behind our best on-policy variant (§6).

In every case the learning problem has the same shape: given pairs (x_t, q) with $\mathbb{E}[e^q | x_t] = e^{V(x_t, t)}$, fit $p = V_\theta(x_t)$. The rest of the paper concerns the two remaining degrees of freedom — the divergence applied to (p, q) , and the sampler that produces the pairs.

3 The problem with exponential-space squared error

The value $V(x, t) = \log \mathbb{E}_{\text{base}}[e^{r(X_1)} | X_t = x]$ is a conditional log-partition function, and the H-martingale property (§2) supplies *conditionally unbiased* targets in *exponential* space: $e^{V(x_t)} = \mathbb{E}[e^q | x_t]$ for a one-step target q (the terminal reward, or a bootstrapped value). A network predicts the value in log space, $p = V_\theta(x_t)$; matching e^p to the unbiased target e^q with squared error gives

$$L_{\text{MSE}}(p, q) = (e^q - e^p)^2, \quad \frac{\partial L_{\text{MSE}}}{\partial p} = 2e^p(e^p - e^q) = 2e^{2p} - 2e^{p+q}. \quad (15)$$

Both tails are pathological, but asymmetrically so (Fig. 1, left).

Over-prediction explodes (recoverable). As $p \rightarrow +\infty$ the gradient grows like $2e^{2p}$ and overflows single precision once $p \gtrsim 44$ nats (the square needs e^{2p}). This is a nuisance but a benign one: the gradient merely needs to be a full-size, correctly-signed step, so saturating it to a large finite value (or clipping) recovers a usable update.

Under-prediction vanishes (unrecoverable). As $p \rightarrow -\infty$ the gradient behaves like $-2e^{p+q} \rightarrow 0$. This is exactly the regime where the model is *most wrong*: it assigns near-zero value $e^p \approx 0$ to a state whose true value e^q is large — a rare, high-reward target it has failed to find. The learning signal there is not merely ill-scaled, it is *absent*, and no amount of gradient clipping can manufacture a gradient that has decayed to zero. In the high-dimensional, signal-sparse regime of §6, where the useful targets are precisely the rare large ones, this is the dominant failure mode.

Why not regress in log space? The obvious escape is to drop the exponentials and fit $(p - q)^2$ directly, which is perfectly conditioned. It is also biased. Squared error in q is minimized by $\mathbb{E}[q | x_t]$, whereas the value is $V = \log \mathbb{E}[e^q | x_t]$; by Jensen the two differ by the log-partition gap

$$V(x_t) - \mathbb{E}[q | x_t] = \log \mathbb{E}[e^{q - \mathbb{E}[q]}] \geq 0, \quad \approx \frac{1}{2} \text{Var}(q | x_t) \text{ for small spread}, \quad (16)$$

and this gap is *largest* in exactly the heavy-tailed, high-dimensional regime the method is meant to handle: it is the quantity that a soft-max is all about, so discarding it discards the signal. Unbiasedness pins us to the exponential-space target; only the divergence applied to it is negotiable. That is the opening the Bregman family exploits (§4).

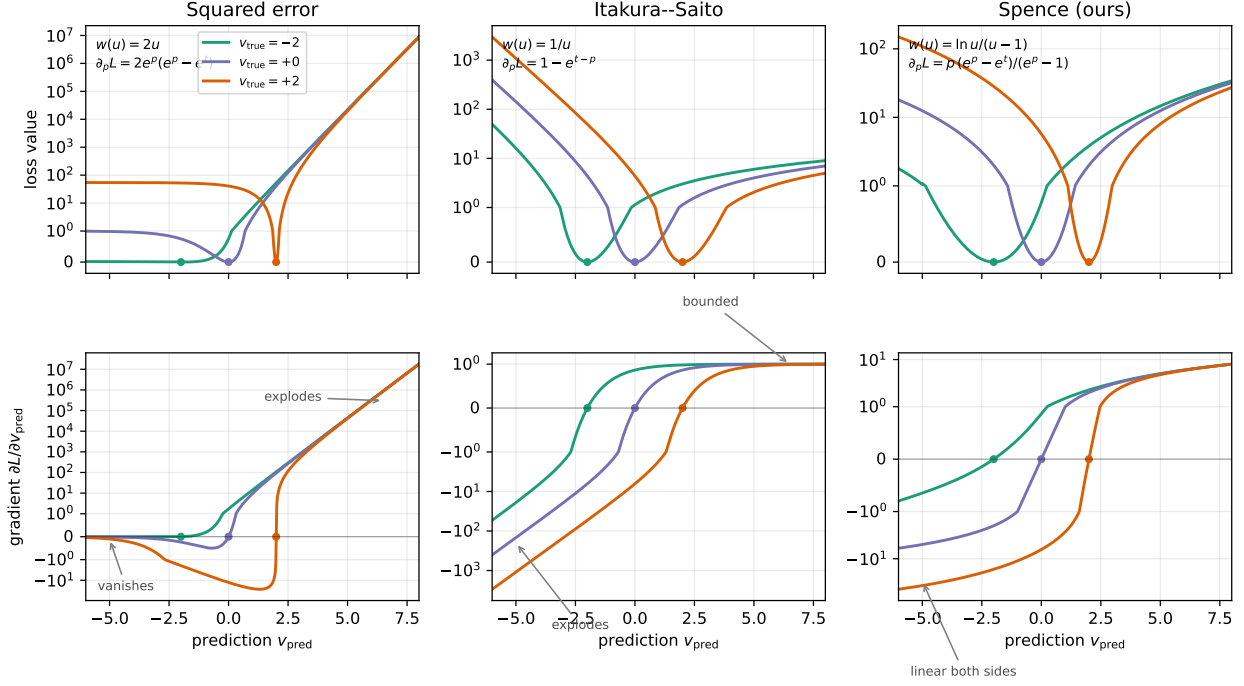


Figure 1: Loss value (top) and gradient $\partial L / \partial v_{\text{pred}}$ (bottom) of $D_F(e^{v_{\text{true}}}, e^{v_{\text{pred}}})$ as functions of the prediction v_{pred} , for three targets $v_{\text{true}} \in \{-2, 0, 2\}$ (dots mark the minimizer $v_{\text{pred}} = v_{\text{true}}$; symmetric-log axes). **Squared error** vanishes for under-predictions ($v_{\text{pred}} < v_{\text{true}}$, gradient $\rightarrow 0$) and explodes for over-predictions. **Itakura--Saito** inverts this: the over-prediction gradient saturates at $+1$, while under-predictions now explode — trading the unrecoverable pathology for a clippable one. **Spence (ours)** is linear in v_{pred} in both tails, so neither side vanishes nor explodes exponentially.

Empirical pathology. Beyond the asymptotics, the finite-precision behavior is severe. In our convergence runs the naive loss skipped or aborted a large fraction of batches on overflow — in one FBRRT run, 18% of batches once predictions reached moderate magnitude — and a *single* non-finite prediction propagated through the shared value network to poison every subsequent batch. We report the full stability audit in §5. A stabilized implementation with the exact gradient of (15) saturated to the float range (our `exp_mse`) removes the overflow and the poisoning, and it is the baseline we compare against throughout, so that the comparison isolates the divergence rather than the arithmetic. What it cannot remove is the vanishing gradient, which is a property of the divergence itself.

4 Bregman divergences for value learning

4.1 Any Bregman divergence learns the value function

For a strictly convex, differentiable potential φ on $(0, \infty)$, the Bregman divergence of a prediction u from a target T is

$$D_\varphi(T, u) = \varphi(T) - \varphi(u) - \varphi'(u)(T - u). \quad (17)$$

Squared error is the special case $\varphi(u) = u^2$, giving $D_\varphi(T, u) = (T - u)^2$. The order of the arguments matters: the prediction must occupy the *second* slot, which is the one whose minimizer is a plain mean.³

Lemma 1 (Bregman divergences are proper losses for the value). *Let the regression target be e^q , with q such that $\mathbb{E}[e^q | x_t] = e^{V(x_t)}$ (the H -martingale target, §2). Parameterize the prediction in log space, $u = e^p$.*

³With the arguments swapped, $\arg \min_u \mathbb{E}[D_\varphi(u, T)]$ solves $\varphi'(u) = \mathbb{E}[\varphi'(T)]$, a quasi-arithmetic mean — the harmonic mean of T for $\varphi = -\ln u$, not $\mathbb{E}[T]$.

Then for any strictly convex φ ,

$$\frac{\partial}{\partial p} D_\varphi(e^q, e^p) = w(e^p) (e^p - e^q), \quad w(u) := u \varphi''(u) > 0, \quad (18)$$

and consequently $\arg \min_p \mathbb{E}_{q|x_t}[D_\varphi(e^q, e^p)] = V(x_t)$.

Proof sketch. Differentiating (17) in u , the $\varphi'(u)$ terms cancel and leave $\varphi''(u)(u - T)$; the chain rule contributes $du/dp = u$. Because (18) is *linear* in the target, taking expectations gives $\mathbb{E}[\partial L/\partial p] = w(e^p)(e^p - \mathbb{E}[e^q | x_t])$, which is negative below e^V and positive above it. Full proof in App. D. $\square$

Two consequences drive the rest of this section. First, the choice of φ does *not* bias the learned value, so it is free to be chosen for *conditioning*. Second, (18) says that a Bregman loss acts on the optimizer only through the scalar *gradient weight* w multiplying the raw exp-space residual $e^p - e^q$. **The design space is exactly the choice of a positive weight function w on $(0, \infty)$** , and the two losses we have met are its opposite extremes.

4.2 Two reference points

Squared error: $w(u) = 2u$. The weight *grows* with the prediction, so $\partial L/\partial p = 2e^p(e^p - e^q)$ explodes like e^{2p} for over-predictions and, because $w(u) \rightarrow 0$ as $u \rightarrow 0$, is driven to zero for under-predictions — the unrecoverable pathology of §3.

Itakura–Saito: $w(u) = 1/u$. Taking $\varphi(u) = -\ln u$ gives $D_{\text{IS}}(e^q, e^p) = e^{q-p} - (q - p) - 1$ and

$$\frac{\partial L_{\text{IS}}}{\partial p} = 1 - e^{q-p}. \quad (19)$$

The weight decays fast enough to cancel the residual entirely on the over-prediction side: as $p \rightarrow +\infty$ the gradient saturates at +1, a bounded, correctly-signed step. On the under-prediction side it now *explodes* like $-e^{q-p}$. Itakura–Saito therefore does not remove squared error’s asymmetry so much as *invert* it, and that is precisely why it helps: it converts the unrecoverable failure (a gradient that has decayed to zero) into the recoverable one (a gradient that is too large, which clipping fixes). It is moreover *scale-invariant* ($D_{\text{IS}}(cT, cu) = D_{\text{IS}}(T, u)$, equivalently (19) depends on $q - p$ alone) and is the negative log-likelihood of a gamma/exponential observation model, giving a natural uncertainty interpretation for V . A useful midpoint, but still one-sided (Fig. 1, middle).

4.3 The Spence loss

Rather than pick a potential and inspect its tails, we ask directly what weight w we want. Writing $u = e^p$ and reading off (18), the two requirements are:

$$p \rightarrow +\infty \ (u \rightarrow \infty) : \quad w(u) u \sim \ln u \quad \implies \quad w(u) \sim \ln u / u, \quad (20)$$

$$p \rightarrow -\infty \ (u \rightarrow 0) : \quad -w(u) e^q \sim e^q \ln u \quad \implies \quad w(u) \sim -\ln u, \quad (21)$$

i.e. in both directions the gradient should grow *linearly in the log-space prediction* $p = \ln u$. The simplest smooth positive function meeting both is

$$\boxed{w(u) = \frac{\ln u}{u - 1}} \quad \iff \quad \varphi''(u) = \frac{\ln u}{u(u - 1)}, \quad (22)$$

which is positive on all of $(0, \infty)$ (both factors change sign at $u = 1$) and extends smoothly through the removable singularity with $w(1) = 1$. It is the reciprocal of the logarithmic mean of 1 and u , and at both extremes it lies between the squared-error and Itakura–Saito weights: the Spence loss interpolates the two reference points rather than adding a third unrelated one. The choice is not unique — (20)–(21) fix only

Table 1: The value-learning loss family, organized by the gradient weight w of (18). Tails give $\partial L/\partial p$ as the prediction $p \rightarrow \pm\infty$ at fixed target q . All three share the same minimizer, $V = \log \mathbb{E}[e^q \mid x_t]$ (Lemma 1); they differ only in conditioning.

loss	weight $w(u)$	under-predict ($p \rightarrow -\infty$)	over-predict ($p \rightarrow +\infty$)	scale-inv.	noise model
squared error	$2u$	<i>vanishes</i> ($\rightarrow 0$)	explodes (e^{2p})	no	Gaussian
Itakura–Saito	$1/u$	explodes ($-e^{q-p}$)	bounded ($\rightarrow +1$)	yes	gamma
Spence (ours)	$\frac{\ln u}{u-1}$	linear ($p e^q$)	linear (p)	no	implicit

the asymptotics — but (22) is the member with a closed-form gradient and a dilogarithm value (App. A). Substituting it into (18) gives

$$\boxed{\frac{\partial L}{\partial p} = p \frac{e^p - e^q}{e^p - 1}} \quad (23)$$

zero iff $p = q$ (the apparent zero at $p = 0$ is cancelled by the pole of $p/(e^p - 1) \rightarrow 1$), with population minimizer V by Lemma 1.

Both tails linear in the prediction. By construction,

$$p \rightarrow +\infty : \quad \frac{\partial L}{\partial p} \rightarrow p, \quad \text{no } e^{2p} \text{ blow-up,} \quad (24)$$

$$p \rightarrow -\infty : \quad \frac{\partial L}{\partial p} \rightarrow p e^q, \quad \text{non-vanishing corrective signal.} \quad (25)$$

We stress what is and is not being claimed. The gradient is linear in the *prediction* p , not in the error $p - q$, so the loss grows like $\frac{1}{2}p^2$ for over-predictions and like $\frac{1}{2}p^2 e^q$ for under-predictions: it is quadratic in p on both sides, but with target-dependent curvature on the under-prediction branch. That e^q factor is the reason the loss is not scale-invariant, and it is deliberate — it up-weights exactly the rare high-value targets that dominate the log-sum-exp and are hardest to find (§6), while still growing only polynomially in p . To our knowledge this is the first Bregman divergence proposed for exp-space value regression with both tails controlled in this sense; the alternative strand of work attacks the same conditioning problem by changing the *objective* rather than the divergence (log-variance and adjoint losses, §8).

Value via the Spence function. The divergence value (used only for monitoring; training uses (23)) is expressible through the Spence function $S(x) = \text{Li}_2(1 - e^x)$, using the dilogarithm identity $S'(x) = -xe^x/(e^x - 1) = x/\text{expm1}(-x)$; hence the name. The full derivation and the numerically stable branch-wise gradient — finite for all float inputs, unlike naive autograd, which returns NaN for $p \gtrsim 88$ in single precision — are in App. A.

Implied observation model. By the duality between regular Bregman divergences and regular exponential families [1], minimizing any D_φ is maximum likelihood under the exponential family whose cumulant is the convex conjugate φ^* ; squared error recovers a Gaussian and Itakura–Saito a gamma. For (22) the conjugate is not a standard named family, so we report the correspondence rather than a closed-form likelihood; the practical content of the weight is the tail behavior above.

5 Off-policy experiments: the loss in isolation

6 Signal sparsity and on-policy learning

7 Experiments

8 Related work

9 Conclusion and limitations

References

- [1] Arindam Banerjee, Srujana Merugu, Inderjit S. Dhillon, and Joydeep Ghosh. Clustering with Bregman divergences. *Journal of Machine Learning Research*, 6:1705–1749, 2005.
- [2] Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh Ranganath. A general framework for inference-time scaling and steering of diffusion models. *arXiv preprint arXiv:2501.06848*, 2025.
- [3] Masatoshi Uehara, Yulai Zhao, Kevin Black, Ehsan Hajiramezanali, Gabriele Scalia, Nathaniel Lee Diamant, Alex M Tseng, Tommaso Biancalani, and Sergey Levine. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv preprint arXiv:2402.15194*, 2024.
- [4] Francisco Vargas, Will Grathwohl, and Arnaud Doucet. Denoising diffusion samplers. *arXiv preprint arXiv:2302.13834*, 2023.
- [5] Qincheng Zhang and Yongxin Chen. Path integral sampler: A stochastic control approach for sampling. *arXiv preprint arXiv:2111.15141*, 2021.

A The Spence loss: derivation and implementation

B On-policy method zoo

C Experimental details

D Additional proofs